

Limpieza y EDA

1.1 Verificación de integridad al cargar los datos

Qué se hizo: al cargar `techmind_dataset_v2.csv`, se agregó una celda que compara el número de filas leídas por dos motores de parseo distintos (C y Python), para detectar si algún texto con saltos de línea mal escapados generaba filas de más o de menos. Se detectó que el archivo original tenía una inconsistencia de 11 registros entre lo que reportaba el notebook y el conteo esperado del CSV. El diagnóstico confirmó que el origen eran fragmentos de documentos en español extraídos por OCR/PDF con saltos de línea sin escapar. Sin esta verificación, la diferencia habría pasado inadvertida.

1.2 Recolección de información (documentación de fuentes)

Qué se hizo: se documentó explícitamente qué aporta cada fuente del dataset: `stackoverflow` (documentación técnica pública, preguntas y respuestas), `medium` (artículos especializados), `freecodecamp_es` (tutoriales), `wikipedia_es` (documentación de referencia). El punto de la rúbrica "Recolección de Información" pedía justificar de dónde salió cada tipo de contenido, no solo tenerlo mezclado en una columna `fuentes` sin explicar su origen.

1.3 Corrección del mapeo de categorías

Qué se hizo: se corrigió un error de tilde en el diccionario de mapeo ("`Programacion General`" sin acento vs. "`Programación General`" con acento en los datos reales). Este error, aunque parecía menor, estaba descartando **5,185 registros completos** (13.5% del dataset) de forma silenciosa, porque el mapeo los convertía en `NaN` y el filtro posterior los eliminaba. Se verificó con una comparación explícita entre las categorías del dataset crudo y las del dataset limpio para confirmar que ya no se pierde ninguna.

1.4 Normalizar en vez de eliminar (valores faltantes)

Qué se hizo: los 535 registros con `palabras_clave` nula ya no se eliminan; se rellenan con "`sin_palabras_clave`". El pedido explícito era conservar la mayor cantidad de información posible para el entrenamiento. Eliminar la fila completa por un solo campo secundario vacío desperdiciaba título y texto, que sí estaban completos.

1.5 Normalización de formato y corrección ortográfica

Qué se hizo: limpieza de texto por regex (URLs, saltos de línea, espacios) preservando caracteres técnicos (`C++`, `C#`, `CI/CD`, `TCP/IP`), más una normalización ortográfica ligera (colapsar letras/signos repetidos, corregir typos comunes). Sin preservar los caracteres técnicos se pierde información relevante para un dataset de contenido de programación (perder el `+` de `C++` lo vuelve irreconocible). La corrección ortográfica se implementó por reglas porque el entorno no tuvo acceso a librerías de diccionario (`pyspellchecker`) sin conexión a internet.

1.6 Filtrado de contenido y eliminación de duplicados

Se descartan textos de menos de 20 palabras (contenido sin valor informativo) y se eliminan duplicados exactos por texto. Son los criterios mínimos de calidad pedidos en "Seleccionar contenido útil" y "Eliminar duplicados" de la rúbrica.

1.7 Balanceo de clases y división train/test — orden corregido

Qué se hizo: originalmente el pipeline balanceaba las clases (con oversampling) y **después** dividía en train/test. Se corrigió el orden: ahora **primero** se divide (`train_test_split` sobre el dataset limpio, no el balanceado) y **después** se balancea, aplicando el oversampling **solo** sobre `train_df`. Con el orden original, copias duplicadas de una misma fila podían caer una en train y otra en test (fuga de datos). Se midió directamente: el 17.3% del test terminaba siendo una copia de algo ya visto en entrenamiento (69.7% en la categoría Seguridad). Con el orden corregido, la fuga se verificó en 0%.

1.8 Análisis Exploratorio (EDA)

Distribución de categorías (original vs. limpia, para detectar pérdidas), valores faltantes, duplicados, sesgos (96% del corpus en inglés, 67% proviene de Stack Overflow), estadísticas descriptivas de longitud, histogramas, frecuencia de palabras, palabras más características por categoría (TF-IDF), balance de clases, y una decisión explícita sobre outliers de longitud. **Por qué cada ajuste específico:**

- El Top-20 de palabras frecuentes inicialmente mostraba solo palabras vacías en inglés (`for`, `you`, `data`) porque la lista de stopwords comparaba en minúsculas contra texto que tenía mayúsculas. Se corrigió pasando el texto a minúsculas antes de comparar, y se amplió la lista con relleno típico de foros técnicos en inglés.
- Se encontró un bug donde `''.join()` pegaba el final de un documento con el inicio del siguiente sin espacio, generando palabras inexistentes; se corrigió a `' '.join()`.
- Se agregó el análisis de palabras características **por categoría** (no solo el Top-20 general) porque es lo que demuestra si las clases son distinguibles entre sí — la evidencia que un modelo de clasificación necesita.
- Se decidió explícitamente **truncar** (no descartar ni dejar completos) los documentos por encima del percentil 99 de longitud (había uno de más de 32,000 palabras), para no perder la señal de categoría que aportan sin que un solo documento extremo pese de más en el entrenamiento.